bookmark-gt

Bookmark manager for Emacs

Daniel M. German

August 12, 2026

Contents

A bookmark manager for Emacs, that replaces the the built-in `bookmark.el` inspired by bookmark+.

It supports most of `bookmark+`'s features: tags, non-file bookmark types, temporary bookmarks, bookmark highlighting, and others.

`bookmark-gt` is integrated with consult, marginalia, and orderless.

Bookmark files created with `bookmark-gt` can be read by the built-in bookmark package and by `bookmark+`. `bookmark-gt` can also read files created by either of those.

1 Install

Using use-package with a local directory

```
(use-package bookmark-gt
  :straight nil
  :load-path "~/.emacs.d/modules/bookmark-gt"
  :bind (("C-x r m" . bookmark-gt-set)         ; supersede built-in bookmark-set
         ("C-x r l" . bookmark-gt-list)        ; supersede bookmark-bmenu-list
         ("C-x r b" . bookmark-gt-jump)        ; fuzzy jump reader
         ("C-x r u" . bookmark-gt-set-url))    ; new: bookmark a URL
  :config
  (bookmark-gt-mode 1))
```

Requires Emacs 30.1+. Soft-deps (loaded when present, package works without them):

- `marginalia` — bookmark completion annotator (tags · type · path).
- `consult` — jump reader UI (falls back to plain `completing-read`).
- `orderless` — enables the `,@type` / `;tag` narrowing particles.
- `browsel` — required only for the browser-tabs feature. See below

2 Getting started

2.1 Master switch: `bookmark-gt-mode`

Global minor mode. When enabled, it installs:

- the interactive tag reader into `bookmark-gt-set`;
- the marginalia annotator into the `bookmark` completion category;
- the temp-bookmark filter on `bookmark-save` (temp records excluded from disk);
- a `:around` on `bookmark--jump-via` that lets URL/browser-tab handlers throw `bookmark-gt-skip-post-hand` to suppress the annotation-buffer popup;
- the file-rename tracker on `rename-file` (see below).

Turning it off removes all of the above.

2.2 Opt-in modes

Three independent global minor modes, off by default. Enable each only if you use the feature.

2.2.1 `bookmark-gt-auto-update-mode`

Bookmarks carrying the `auto-update` alist property get their position refreshed to the visiting buffer's current point on every idle tick. Useful for e-readers, long-form documents, `pdf-tools` page tracking.

Triggers: an idle timer (`bookmark-gt-auto-update-interval`, default 60 s) plus `kill-buffer-hook` and `window-state-change-hook`. Also registered into the ephemeral refresh hook, so `g` in the list buffer or opening the jump reader triggers an immediate refresh.

Toggle the property on the row at point with `^` in the list buffer, or `M-x bookmark-gt-auto-update-toggle NAME`.

2.2.2 `bookmark-gt-default-tags-mode`

Seeds the tag reader on `bookmark-gt-set` with context-appropriate tags. Configured via a small DSL:

```
(setq bookmark-gt-default-tags "work")                    ; single tag
(setq bookmark-gt-default-tags '("work" "urgent"))        ; list
(setq bookmark-gt-default-tags                            ; function
      (lambda (record)
        (cond ((string-prefix-p "/proj/" (bookmark-gt-filename-of record))
               '("project"))
              ((eq (bookmark-gt-handler-type record) 'url)
               '("web"))
              (t nil))))
```

Errors in the function form are demoted to a warning; nothing is force-applied. The user always sees the defaults as prefill and can edit or clear them.

2.2.3 `bookmark-gt-browsel-tabs-mode`

Exposes the browser's open tabs as bookmarks (temporary — not saved to disk). Requires `browsel` to be installed and connected.

Refresh is on-demand, not on a timer:

- immediate refresh on mode enable;
- before every `bookmark-gt-jump` read;
- before opening the list buffer;
- on `g` (revert) in the list buffer;
- `M-x bookmark-gt-browsel-tabs-refresh` for a manual refresh.

Turning the mode off clears our tab records and unregisters the refresher. Records owned by other browsel-related packages (`browsel-tab-manager-bookmark-jump` etc.) are left alone.

The refresh loop suppresses per-tab hook execution and auto-save inside the batch — with 100+ tabs the whole refresh takes ~50 ms total (30 ms fetch + 20 ms store).

Customizable:

- `bookmark-gt-browsel-tabs-browsers` — list of browser clients to query, nil = all connected.
- `bookmark-gt-browsel-tabs-filter` — nil / regexp / predicate applied per tab before storing.

3 User interface

3.1 The `*Bookmarks-gt List*` buffer

`M-x bookmark-gt-list` opens the buffer. It is a `tabulated-list-mode` derivative — click a column header to sort, press `S` for column-at-point sort, and `s` (specific to bookmark-gt) to cycle the sort column through the marker and data columns.

3.1.1 Columns

Col	Header	Meaning
0	`*`	Selection: `*` if marked with `m`, `D` if flagged for deletion with `d`, blank else
1	`^`	Auto-update: `^` if the record carries the `auto-update` prop, blank else
2	`t`	Temp: `t` if the record carries `bmkp-temp`, blank else
3	Name	Truncated at `bookmark-gt-list-name-width` (default 32) with an ellipsis
4	Type	From the handler registry — File, URL, EWW, Dired, Info, Org, PDF, BrowserTab, ...
5	Group	Aggregate category — Web, File, Doc, Code, Media, Other
6	Tags	`,`-joined, truncated at `bookmark-gt-list-tags-width` (default 24)
7	Loc	`filename`, or the URL if non-file

The Name of a bookmark whose file no longer exists on disk renders with a strikethrough (`bookmark-gt-face-missing-file`). Remote (Tramp) paths are not probed — the existence check is local only.

3.1.2 Keys

Key	Action
`RET`	Jump
`o`	Jump in another window
`C-o`	Same as `o`
`v`	Preview in another window (leaves point in the list)
`m`	Mark row (numeric prefix: mark N rows down)
`d`	Flag row for deletion
`x`	Delete every flagged row (prompts for confirmation)
`u`	Unmark row
`U`	Unmark every row
`^`	Toggle `auto-update` on row
`T`	Toggle temp on row
`H`	Toggle display of temporary bookmarks in this buffer
`r`	Rename bookmark
`R`	Relocate (change filename or URL)
`t`	Edit tags
`a`	Edit annotation
`/`	Filter (choose by-type / by-tag / by-name-regexp / unfilter)
`s`	Cycle sort column
`g`	Revert (refresh ephemeral sources first)
`i`	Inspect record (pretty-print alist in a popup)
`q`	Quit

Selection marks are stored in a buffer-local hash keyed by record cons identity — they survive sort, revert, and after-hook re-renders.

3.2 Jump reader: `bookmark-gt-jump`

Uses `consult--read` when consult is loaded, falls back to `completing-read` otherwise. Both accept two narrowing particles when orderless is loaded:

- `,@TYPE` — narrow to bookmarks of TYPE (`url`, `eww`, `file`, `dired`, `info`, `org`, `pdf`, `browser-tab`, `magit`, `help`, `image`, `epub`, or any registered/derived type).
- `;TAG` — narrow to bookmarks carrying TAG.

Mid-read tag filter:

- `M-t` — add a tag filter and restart the read. Active filters are shown as labels in the prompt.
- `M-d` — pop the most recent tag filter.
- `M-D` — clear every active tag filter.

3.2.1 Elisp signatures

All three entry points take keyword arguments only:

```
(cl-defun bookmark-gt-jump
    (&key bookmark display-function bookmarks-list bookmarks-filter
          sort-by preselect))

(cl-defun bookmark-gt-jump-other-window
    (&key bookmark bookmarks-list bookmarks-filter sort-by preselect))

(cl-defun bookmark-gt-jump-tagged
    (&key tag bookmark bookmarks-list bookmarks-filter sort-by preselect))
```

- `:BOOKMARK` — bookmark name or record. When nil, prompt.
- `:DISPLAY-FUNCTION` — a function of one argument (a buffer); passed to 'bookmark-jump' as its display func.
- `:BOOKMARKS-LIST` — alist to use as the candidate pool. Defaults to 'bookmark-alist'. Ignored when `:BOOKMARK` is supplied.
- `:BOOKMARKS-FILTER` — predicate on a record; a candidate is kept when the predicate returns non-nil. Ignored when `:BOOKMARK` is supplied.
- `:SORT-BY` — `mru` (by `last-visited` desc), `visits` (by `visits` desc), or nil (preserve pool order). Defaults to `bookmark-gt-jump-default-sort` (`mru` by default).
- `:PRESELECT` — positive integer. When set (and vertico is loaded), the reader positions the cursor on the N-th candidate via a one-shot minibuffer-setup hook. Useful for a "jump back to previous" workflow: with MRU sort and `:preselect 1`, RET selects the second-most-recent record.
- `:TAG` (required for `-tagged`) — string seeding the reader's active tag filter.

Candidate row layout is customizable via `bookmark-gt-jump-candidate-format-function` (default: `bookmark-gt-jump-candidate-default`). A custom function receives the record and must return a propertized string with the `bookmark-gt-name`, `consult--type`, and (optional) `bookmark-gt-particles` text properties — see the default function's docstring for the contract.

Examples:

```
;; Jump directly to a known bookmark
(bookmark-gt-jump :bookmark "my-bookmark")

;; Jump with a custom display function (other-frame etc.)
(bookmark-gt-jump :display-function #'switch-to-buffer-other-frame)

;; Prompt but only among file bookmarks
(bookmark-gt-jump :bookmarks-filter #'bookmark-gt-handler-file-p)

;; Prompt among a custom subset alist
(bookmark-gt-jump :bookmarks-list my-project-bookmarks)

;; Prompt among project files tagged "urgent"
(bookmark-gt-jump-tagged :tag "urgent"
                         :bookmarks-filter #'bookmark-gt-handler-file-p)
```

3.3 Non-file bookmark types

3.3.1 URL bookmarks — `bookmark-gt-set-url`

Interactive:

```
M-x bookmark-gt-set-url RET
URL: https://example.org RET
Bookmark name: Example RET
```

Elisp:

```
(bookmark-gt-set-url "https://example.org" "Example" '("web" "reference"))
```

The URL bookmark's handler (`bookmark-gt-handler-url-jump`) opens the URL via `browse-url`. It also throws `bookmark-gt-skip-post-handler` so the built-in post-handler flow does not pop up an annotation buffer moving focus from the browser back to Emacs.

3.3.2 Dired bookmarks — `bookmark-gt-handler-dired-jump`

The built-in `dired.el` does not provide a bookmark handler. bookmark-gt provides one that calls `(dired FILENAME)`. Records migrated from bookmark+ (`bmkp-jump-dired`) are still recognized as Dired type via the registry alias; you can migrate them off bookmark+ by setting their `handler` property to `bookmark-gt-handler-dired-jump`.

3.3.3 Function bookmarks — `bookmark-gt-set-function`

A function bookmark stores a callable (a symbol or lambda) under the `function` alist key. Jumping invokes the callable.

Interactive:

```
M-x bookmark-gt-set-function RET
Function bookmark name: rebuild-project RET
Function to call on jump: my-project-rebuild RET
```

Elisp:

```
(bookmark-gt-set-function "rebuild" #'my-project-rebuild)
(bookmark-gt-set-function "flip-theme"
                          (lambda () (my-toggle-theme)))
```

3.3.4 Sequence bookmarks — `bookmark-gt-set-sequence`

A sequence bookmark stores a list of bookmark names under the `sequence` alist key. Jumping visits each in order via `bookmark-jump`.

Interactive: prompts for the sequence name, then reads existing bookmark names one at a time until you enter empty input.

Elisp:

```
(bookmark-gt-set-sequence "morning-workspace"
                          '("inbox" "todo.org" "calendar"))
```

4 Setting bookmarks

4.1 Tag reader

`bookmark-gt-tags-read` prompts for tags one at a time. Each prompt is a plain `completing-read` against every tag that currently appears in `bookmark-alist`, ordered most-recently-used first (based on `bookmark-gt-tags-history`; completion history is the MRU signal).

Terminators:

- Empty `RET` — finish, return the accumulated list.
- `M-RET` — submit the current input (if non-empty), then finish.

Any non-empty `RET` accepts the current tag and prompts for another. The running accumulator is shown in the prompt as ~Tags [foo,bar] (M-RET or empty RET to finish): ~.

The reader is invoked by `bookmark-gt-set` when `bookmark-gt-prompt-for-tags-flag` is non-nil (default) and seeds itself with any tags provided by `bookmark-gt-default-tags-mode` or another `bookmark-gt-set-tag-reader-hook` contributor.

4.2 Same-name policy

The name a record is stored under is resolved in this order:

1. `C-u M-x bookmark-gt-set` (or NO-OVERWRITE non-nil from Lisp) always disambiguates with a `<N>` suffix — the escape-hatch that matches built-in `bookmark-set`.
2. If the name collides with an existing record that has the same handler AND the same filename, `bookmark-gt-same-name-overwrite` (default `t`) replaces the old record in place — the common case of "update this bookmark's location."
3. Otherwise, `bookmark-gt-allow-duplicate-names` (default `t`) stores the record under the literal name, so two bookmarks in different files (or with different handlers) can share the same name — e.g. a `todo` bookmark in each of several project files.
4. With both flags off, any collision falls back to the `<N>` suffix.

Trade-off on rule 3: name-based lookups (`bookmark-jump NAME`, `bookmark-get-bookmark NAME`) use the built-in `assoc`, which returns the first match — so when two records share a name, only one is reachable by name. The bookmark-gt list buffer and jump reader iterate over the alist directly, so both records appear as distinct rows/candidates with their Location column distinguishing them. Set `bookmark-gt-allow-duplicate-names` to nil if name-based lookup is more important than short shared names.

4.3 Relocate: change a bookmark's target

`M-x bookmark-gt-relocate NAME` changes where an existing bookmark points, dispatching on the record's shape:

- File records (a `filename` key) prompt via `read-file-name`; default is the current filename so a rename is a matter of editing. `position` is left unchanged — the common trigger is a file rename, and the offset is still valid.

- URL records (a `url` key: URL bookmarks, browser-tab bookmarks) prompt for the new URL via `read-string`.
- Records with neither key raise a `user-error`.

In the list buffer, `R` relocates the row's record. The after-hook fires so the list, highlight overlays, and any other observer refresh automatically.

For a genuinely new position within a different file, jump to the target, place point, and re-invoke `bookmark-gt-set` with the same name — the same-name-overwrite policy (above) updates `position` in place.

5 In-file behaviors

5.1 In-buffer cycling

Two autoloaded commands iterate over the bookmarks whose file matches the current buffer's file, sorted by position:

- `M-x bookmark-gt-cycle-next` — jump to the next bookmark position after point.
- `M-x bookmark-gt-cycle-prev` — jump to the previous one.

Both wrap around at the ends of the sequence. Print the bookmark's name via `message` so the destination bookmark is identified. No overlay dependency — cycling uses record data directly, so it works whether or not `bookmark-gt-highlight-enable` is on.

Records without a numeric `position` (e.g. Dired records) are skipped; bookmarks pointing at other files are ignored.

Convenient bindings (in your init):

```
(global-set-key (kbd "C-c n") #'bookmark-gt-cycle-next)
(global-set-key (kbd "C-c p") #'bookmark-gt-cycle-prev)
```

5.2 Per-bookmark highlighting

When `bookmark-gt-mode` is on, every file-visiting buffer gets an overlay per bookmark that points at that file. Overlays use `bookmark-gt-face-highlight` (inherits `hl-line` by default) and span:

- The bookmark's line, for point bookmarks.
- The whole region (from `position` line-start to `end-position` line-end), for region bookmarks.

Hovering shows the bookmark name via `help-echo`.

Triggers:

- `find-file-hook` creates overlays as files are opened.
- `bookmark-gt-set-after-hook` refreshes overlays after any mutation. Targeted single-file refresh when the mutation affects a specific record; full refresh only when a batch operation runs the hook with the `nil` sentinel.

- `bookmark-after-jump-hook` records the point after the jump in a buffer-local hash so records that carry no numeric `position` (e.g. org-heading bookmarks, whose target is resolved by `org-bookmark-heading-jump` at jump time) still get an overlay after the user visits them. The recorded position is consulted by `bookmark-gt-highlight--effective-position` as a fallback when `position` is absent.

Customization:

- `bookmark-gt-highlight-enable` (default `t`) — opt out. Set to nil to suppress highlighting even while the mode is on.
- `bookmark-gt-face-highlight` — face for the overlay. Rebind to a subtler / stronger color as you prefer.

Overlays are process-scoped display state; they are removed when the buffer is killed, and are cleared entirely when `bookmark-gt-mode` turns off. Nothing is written to disk. The post-jump position hash is likewise buffer-local and transient.

5.3 Region bookmarks

When `bookmark-gt-mode` is on and the region is active on `M-x bookmark-gt-set`, the record captures both anchors plus context strings so the region can be re-selected on jump:

- `position` = `(region-beginning)`
- `end-position` = `(region-end)`
- `front-context-region-string` = `N` characters preceding `end-position` (for re-anchoring)
- `rear-context-region-string` = `N` characters following `end-position`
- `front-context-string` / `rear-context-string` = the built-in standard re-anchor strings around the start

Alist-key names match bookmark+ so records round-trip losslessly. The built-in `bookmark.el` ignores the extra keys and jumps to `position` only.

On jump, a hook on `bookmark-after-jump-hook` pushes the mark at `end-position` (adjusted by the delta between the record's raw `position` and the actual point after built-in re-anchoring) and activates it — the region reappears highlighted.

Customization:

- `bookmark-gt-use-region` (default `t`) — opt out of capture and restore. Records that already carry region info keep it; this flag only gates the set/restore paths.
- `bookmark-gt-region-context-size` (default 40) — chars of context captured around `end-position`.

6 Trackers

6.1 Visit tracker

When `bookmark-gt-mode` is on, an entry on `bookmark-after-jump-hook` increments each jumped bookmark's `visits` property by one and refreshes its `last-visited` prop to the current time. URL

and browser-tab handlers throw `bookmark-gt-skip-post-handler` (which bypasses the hook), so they call `bookmark-gt-record-visit` at the end of their handler body — same effect.

Enables `:sort-by 'mru` and `:sort-by 'visits` in the jump reader for every bookmark type without user action.

Visit tracking does NOT bump `bookmark-alist-modification-count`, so every jump does not force an auto-save. Changes persist to disk at the next explicit `bookmark-save` (or on Emacs exit).

6.2 File-rename tracker

When `bookmark-gt-mode` is on, an `:around` on `rename-file` rewrites any bookmark whose `filename` matches the source path to the destination. Intercepts Dired renames (`R`) as well, since Dired uses `rename-file` internally.

Two guards:

- `bookmark-gt-track-renames` — user opt-out, default `t`. Set to nil to disable tracking entirely; the flag is read at call time so changes take effect immediately.
- `backup-file-name-p` on the destination — always skipped. Emacs's default save mode renames ORIG → ORIG~ before writing new content; without this guard the bookmark would follow the rename and end up pointing at your backup file. (Bookmark+'s tracker has this bug; ours does not.)

Directory renames update only bookmarks whose `filename` equals the directory exactly. Children are NOT rewritten.

7 Extension API

7.1 Handler registry

`bookmark-gt-handler-alist` is a flat alist keyed by handler symbol. Each entry:

```
(HANDLER . (:type TYPE-SYMBOL :name STRING :group GROUP-SYMBOL
            :face FACE :narrow-char CHAR :doc STRING))
```

Multiple handler symbols may share the same `:type` (URL has three handler-symbol aliases, browser-tab has three, etc.). Entries are included for: file, URL, EWW, Dired, Info, Org, PDF, browser-tab, Magit, Help, Image, EPUB (nov.el).

Register a new handler (or an alias for an existing type):

```
(bookmark-gt-handler-register
 '(my-custom-jump my-legacy-jump-handler)
 (list :type 'custom :name "Custom" :group 'other
       :face 'bookmark-gt-face-file :narrow-char ?c
       :doc "My custom bookmark type."
       :preview #'my-preview-fn))   ; optional
```

The optional `:preview` key overrides what `bookmark-gt-list-preview` (bound to `TAB` in the list buffer) does for that handler. Called with the record; runs inside `save-selected-window` so it can pop up a preview buffer without moving point away from the list. When absent, preview falls back to `bookmark-jump-other-window` on the record's own jump handler — the current default.

Useful for e.g. rendering a URL in an EWW buffer as preview while `RET` still opens the external browser.

Unknown handler symbols are auto-classified via a fallback that strips common suffixes (`-bookmark-jump-handle` `-bookmark-jump`, `-jump-handler`, `--handle-bookmark`) and title-cases the leading segment. So `foo-bookmark-jump` shows up as type "Foo" without any registration. Its group defaults to `other`.

Type predicates: `bookmark-gt-handler-file-p`, `-url-p`, `-eww-p`, `-dired-p`, `-info-p`, `-org-p`, `-pdf-p`, `-browser-tab-p`. Each does `(eq (bookmark-gt-handler-type record) 'TYPE)`.

7.2 Group registry

Groups aggregate multiple types so users can filter or narrow by an aggregate category (e.g. "web" covers URL + EWW + browser-tab) rather than one type at a time.

Groups are DERIVED from the handler registry at classification time — they are not stored on the record, never written to disk.

Six built-in groups are provided:

Symbol	Name	Narrow char	Types
`web`	Web	`w`	url, eww, browser-tab
`file`	File	`f`	file, dired
`doc`	Doc	`d`	info, org, pdf, help, epub
`code`	Code	`c`	magit
`media`	Media	`m`	image
`other`	Other	`o`	catch-all for unregistered handlers

`bookmark-gt-group-alist` is the registry (group symbol → :name/:narrow-char/:doc plist). Register a new group:

```
(bookmark-gt-group-register
 'science '(:name "Sci" :narrow-char ?s
            :doc "Science / research bookmarks."))
```

The list buffer shows a `Group` column (after `Type`). The consult jump reader narrows by group — pressing `w SPC` shows every web bookmark regardless of type. Per-type filtering remains available via the `,@TYPE` orderless particle. Filter in the list via `/` → `by-group`.

Accessor: `(bookmark-gt-handler-group RECORD)` returns the group symbol (or `other` when the handler is unregistered).

7.3 Extension points

Every non-trivial mechanism has a documented extension point.

- `bookmark-gt-set-name-reader-hook` — refine the bookmark name before the interactive prompt. `(default-name context)` → string (first non-nil wins).
- `bookmark-gt-set-tag-reader-hook` — chained: each hook gets `(record seed-tags)` and returns new seed-tags. This is where `bookmark-gt-default-tags-mode` and the interactive tag reader register themselves.
- `bookmark-gt-set-after-hook` — runs on every mutation. The list buffer refreshes here; other observers can as well.

- `bookmark-gt-ephemeral-refresh-hook` — runs before jump, list open, and list revert. Ephemeral sources (`bookmark-gt-browsel-tabs-mode`, `bookmark-gt-auto-update-mode`) register into it.
- `bookmark-gt-filter-alist` — filter registry for the list buffer's `/` command.
- `bookmark-gt-sort-alist` — sort registry (includes name and type entries).

8 Compatibility

8.1 Data compat with bookmark+

By design, bookmark-gt reads and writes alist entries under the same keys bookmark+ uses so files round-trip both ways:

Key	Meaning
`tags`	List of tag strings
`bmkp-temp`	Temp-record marker
`auto-update`	Auto-tracking flag
`visits`	Visit count
`last-visited`	Last visit timestamp

Two shape differences we accommodate on read:

- Bookmark+ writes the placeholder string `" - no file -"` into `filename` for non-file records. `bookmark-gt-filename-of` treats it as nil. Bookmark-gt records omit `filename` entirely for non-file records.
- Bookmark+ URL records store the URL under `location`. Bookmark-gt writes it under `url`. Both are read by `bookmark-gt-url-of`.

9 Reference

9.1 Selected customization

Variable	Purpose
`bookmark-gt-list-name-width`	List Name-column width (32)
`bookmark-gt-list-type-width`	List Type-column width (12)
`bookmark-gt-list-tags-width`	List Tags-column width (24)
`bookmark-gt-list-buffer-name`	List buffer name
`bookmark-gt-list-selection-mark`	Mark char (default `?*`)
`bookmark-gt-list-deletion-mark`	Deletion flag char (default `?D`)
`bookmark-gt-jump-name-max-width`	Jump reader name truncation (40)
`bookmark-gt-jump-default-sort`	Jump reader default sort ('mru)
`bookmark-gt-auto-update-interval`	Auto-update idle seconds (60)
`bookmark-gt-track-renames`	Rename tracker on/off (t)
`bookmark-gt-default-tags`	Default-tags DSL value
`bookmark-gt-prompt-for-tags-flag`	Prompt for tags on set (t)
`bookmark-gt-same-name-overwrite`	Overwrite same-name+handler+file (t)
`bookmark-gt-allow-duplicate-names`	Allow two records to share a name (t)
`bookmark-gt-highlight-enable`	Per-bookmark overlays (t)
`bookmark-gt-browsel-tabs-browsers`	Which browser clients to query
`bookmark-gt-browsel-tabs-filter`	Per-tab accept filter

10 Acknowledgements

bookmark-gt owes a substantial debt to Drew Adams and his bookmark+ package, both in design and in code. Nearly every feature bookmark-gt provides — tags, non-file bookmark types (URL, EWW, region bookmarks), temporary bookmarks, visit tracking, per-bookmark highlighting, tag-driven filtering, the file-rename tracker — was first introduced in bookmark+.

Concretely reused:

- **Handler bodies.** The URL bookmark handler (`bookmark-gt-handler-url-jump`) and the Dired handler (`bookmark-gt-handler-dired-jump`) were copied from bookmark+'s corresponding handlers (`bmkp-jump-url-browse` and `bmkp-jump-dired`) and lightly adapted — same jump semantics, same record-shape expectations, minor changes to fit bookmark-gt's registry and its `bookmark-gt-skip-post-handler` throw convention.
- **Record schema.** The on-disk alist keys — `tags`, `bmkp-temp`, `auto-update`, `visits`, `last-visited`, `location` (URL alias), `front-context-region-string` / `rear-context-region-string` (region bookmarks), and the `" - no file -"` filename placeholder for non-file records — are all adopted from bookmark+'s format verbatim so bookmark files round-trip in either direction without data loss.

`bookmark-gt` is a rewrite with different architectural choices (no advice on other packages, a single registry per axis, tabulated-list-mode instead of custom rendering), not a fork. But the user-visible features, the design ideas, and the handler code included in this package all originate from Drew's work.

11 License

GPL-3.0-or-later. See `LICENSE`.